lecture 18
Tarjan's algorithm to compute strongly connected components

Don Knuth
- explore directed graph
- does topsort as byproduct

Given a directed graph:
- MST are undirected

Recall for DAG (Directed Acyclic Graphs),
there is a topological sorting

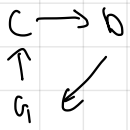
- only visit node once all predecessors have been visited
- the naive topsort algorithm left out cycles

reverse post order is also a topological ordering

Question: what if DFS on a cyclic graph?
- topological order on a cyclic graph

Def: Nodes v and u are strongly connected
if there is a cycle from v to u and back or u=v

v ≈ u



D can get to G
G can get to D

D ≈ G, D ≈ D

every node is strongly connected to itself

Lemma: Any directed graph can be partitioned into strongly connected components. SCCs

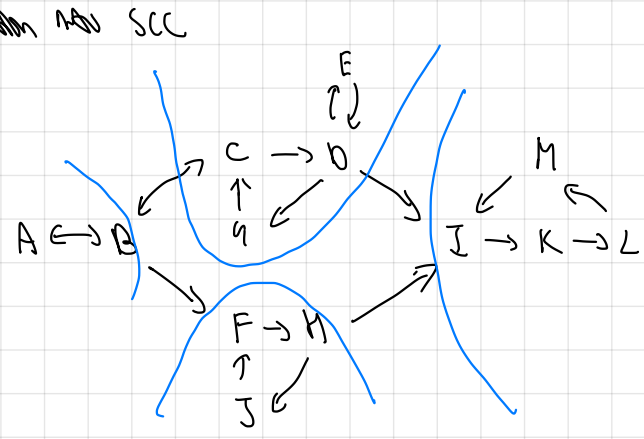
Where nodes u and v are in the same SCC if and only
u is strongly connected to v

Within a SCC, everyone is strongly connected with each other.

all nodes will land in a SCC will land in exactly 1
a SCC can just be 1 node

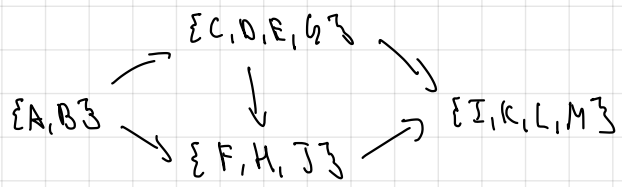
---

partition into SCC



- each node in a SCC can get to every other node in the SCC
- if there is overlap between multiple SCCs, you can just merge them together. the union is also strongly connected

condensation graph
- collapse all strongly connected components together
- turn each SCC into a node
- turn edges between SCCs into edges between nodes



- the issue with topological sort is cycles
- by identifying strongly connected components, we identify and collapse all cycles

- condensation graph has no cycles because if there was a cycle there must be another SCC. However, we already identified and collapsed all of them.

- If every node has outgoing edges, you can make arbitrary long paths
- so eventually, you will repeat a node, making a cycle

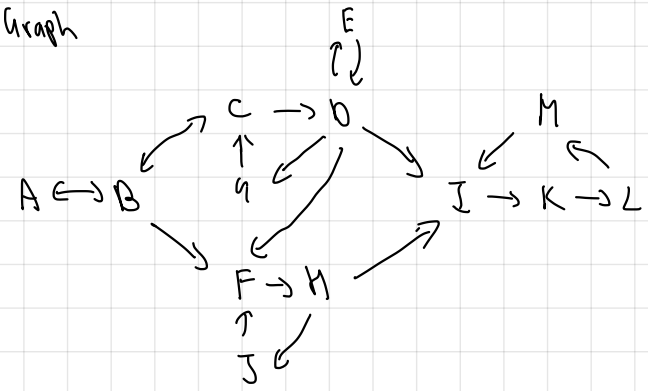
- condense should have picked it up already

- a DAG has at least 1 sink node
- when you walk a DAG with DFS, you will eventually stop at the sink node

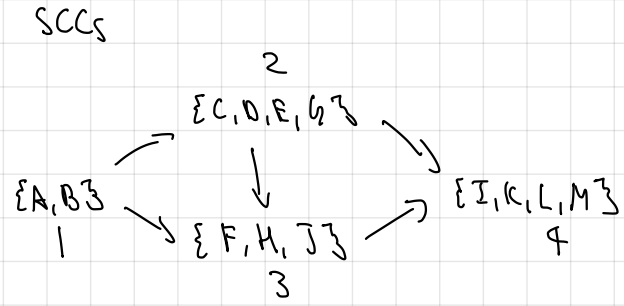
the condensation graph will have a sink node

Run DFS
- when choosing outgoing edge to visit, pick in alphabetical order

Graph



SCCs



Pre Order DFS and note SCC
DFS start at node A

```
def visit(node):
    mark node visited
    pre_order.append(node)
    for other_node in node.outgoing
        if other_node not visited:
            visit(other_node)
```

Pre: A, B, C, D, E, F, H, I, K, L, M, J, G
SCC: 1 1 2 2 2 3 3 4 4 4 4, 3, 2

notice:
- the entire SCC 4 was visited at once
- the sink SCC only has edges internally to the SCC
- will visit whole SCC and not go anywhere

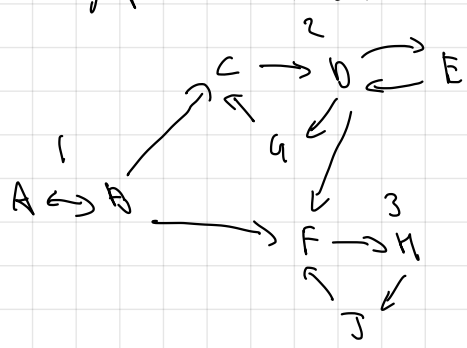
Important: sink component will be visited at once

has to be a sink component since the condensation graph is a DAG

if we keep track of when we enter and leave each SCC, we can identify the sink SCC

1. Tarjan's algorithm can figure out when we see a sink component
2. then it will ignore the sink component in future operations
3. the graph without the sink SCC will have another sink component

Identify strongly connected components

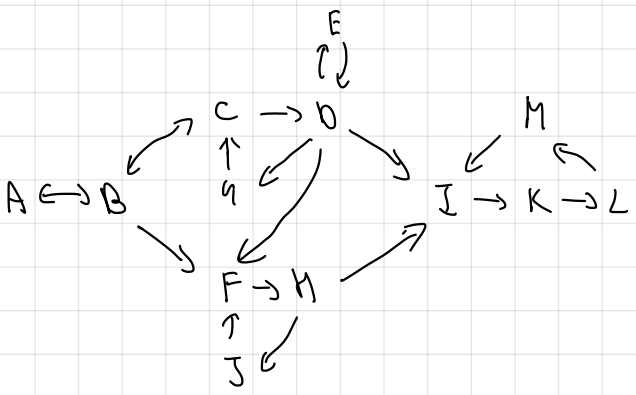


Tarjan's algorithm will figure out when we leave a sink component.

Once we leave, pretend it isn't there and find the next sink component

Pre: A B C D E F H J G
SCC: 1 1 2 2 2 3 3 3 2

Run Tarjan's Algorithm



Keep track
- order of visiting nodes
- number: postorder / pre order list
- low: out of all things we have seen this node reach, what's the number of the earliest one

```
clock ← 0
def visit(node):
    node.number ← clock                  # set number when visited
    clock ← clock + 1
    node.low ← node.number               # initially can reach itself

    mark node visited
    pre_order.append(node)
    for other_node in node.outgoing
        if other_node.number = None:     # means not visited
            visit(other_node)

        if other_node.scc_root ≠ not None    # if other node is part of a sink SCC
            ignore
        elif other_node.low < node.low:      # low is the smallest number node
            node.low ← other_node.low        # I can reach, I can reach an
                                             # even smaller number node
    # found a sink after checking edges
    if node.low == node.number
        while True:
            n ← preorder.pop_back()
            n.scc_root = node
            if n == node:
                break

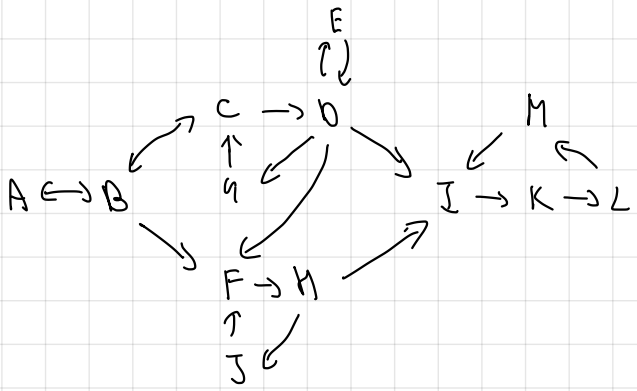
            .
```

Stop when you finish visiting the sink component

Node	number	low	scc_root	* If will loop back to a
A	0	0		If the low value is the same as number, you are a sink if the other node's low is your number
B	1	~~1~~ 0		use C's low
C	2	2		use L's low
D	3	3		use M's low
E	4	~~4~~ 3		I can reach I, with low
F	5	5		
G				
H	6	6		number == low, found a sink
I	7	7	I	nothing lower — this is the condition tarjan's algorithm checks
J				
K	8	~~8~~ 7	I	use L's low
L	9	~~9~~ 7	I	use K's low
M	10	~~10~~ 7	I	use I's low

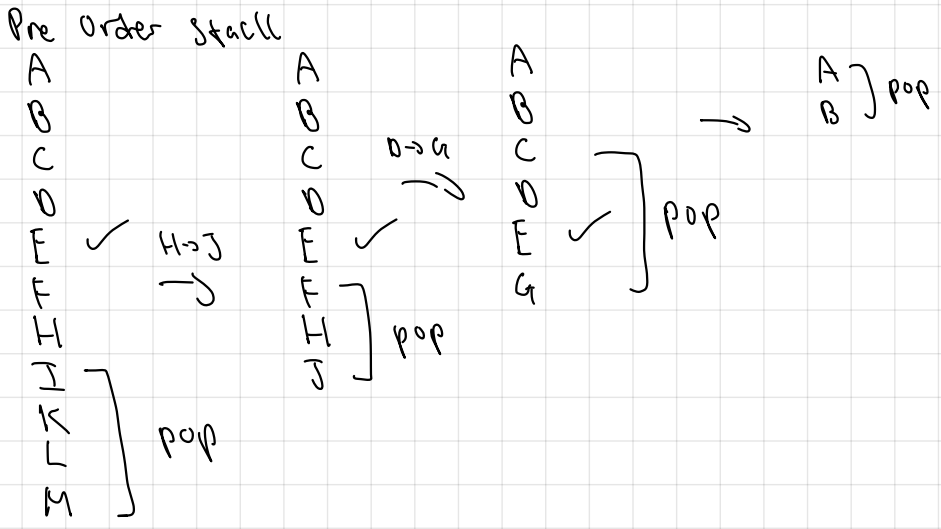
When you find a sink, pop nodes from preorder
- label them with the strongly connected component root

Keep going, but as if I SCC never existed



Node	number	low	scc.root
A	0	0	A
B	1	~~X~~ 0	A
C	2	2	C
D	3	~~3~~ 2	C
E	4	~~4~~ 3	C
F	5	5	F
G	12	2	C
H	6	~~6~~ 5	F
I	7	7	I
J	11	5	F
K	8	~~8~~ 7	I
L	9	~~9~~ 7	I
M	10	~~10~~ 7	I

Pre Order Stack



Running Time: $O(n+m)$